

CM04 - Primitive Data Types

Instructions, representations, objects, operators, conversions, lifetime

Louis Ledoux

2026-07-05

Instructions and Comments

Instructions and Comments

Live pacing: about 8 min

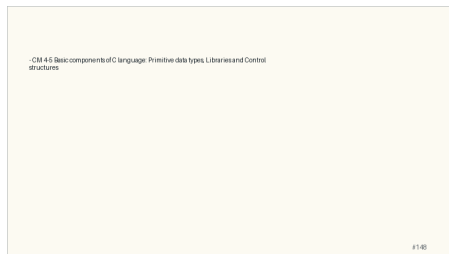
Question. What is the role of Instructions and Comments in the course story?

Core claim. Instructions and Comments is one step in the path from source text to reliable execution.

Target capability. Students can connect the topic to a concrete command, program state, or memory state.

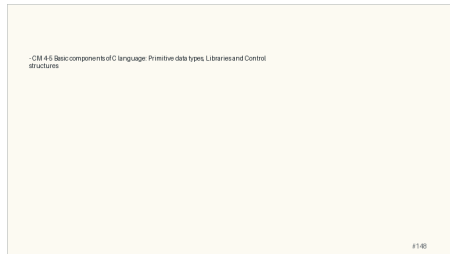
Main Path

- ▶ Start from the question: What is the role of Instructions and Comments in the course story?
- ▶ Build the model: Instructions and Comments is one step in the path from source text to reliable execution.
- ▶ End with a checkable action: Students can connect the topic to a concrete command, program state, or memory state.
- ▶ Keep only this slide on the horizontal path during the live lecture.



Worked Example

- ▶ Use this as the live trace: Use one short trace, then ask students which state changed and which state did not.
- ▶ Representative source slide: 148
- ▶ Ask students to predict the next state before revealing the explanation.
- ▶ Then connect the result back to the core claim.



Anecdote Or Motivation

Keep the discussion anchored in observable behavior: command output, compiler message, or memory drawing.

- ▶ Use this only if students need a reason to care.
- ▶ If the room is already following, skip down to the check question.

Check Question

- ▶ Ask for a prediction, not a definition.
- ▶ Require a short justification: command trace, memory drawing, type rule, or file state.
- ▶ If more than a third of the room hesitates, go down to the source backup.

Instructions & Comments	
- Instruction is any expression finishing with;	
- Block of instructions	
- {	

#149

Backup Index

Original slides 148-149

- CM 4-5 Basic components of C language: Primitive data types, Libraries and Control structures

Backup: CM 4-5 Basic components of C language: Primitive data types, Libraries and Control structu

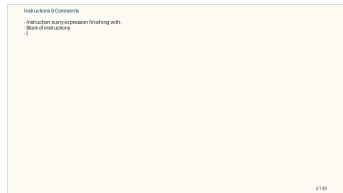
Original slide 148

- CM 4-5 Basic components of C language: Primitive data types, Libraries and Control structures

Backup: Instructions & Comments

Original slide 149

- ▶ Instruction is any expression finishing with ;
- ▶ Block of instructions:
 - ▶ {
 - ▶ list of instructions
 - ▶ }
- ▶ Comments
 - ▶ /* this is a comment */
 - ▶ // this is one line comment
- ▶ The language C does not support disambiguation
- ▶ The keywords of the language are reserved and cannot be used for naming variables and functions.



Value Representation and Data Objects

Value Representation and Data Objects

Live pacing: about 8 min

Question. What is the role of Value Representation and Data Objects in the course story?

Core claim. Value Representation and Data Objects is one step in the path from source text to reliable execution.

Target capability. Students can connect the topic to a concrete command, program state, or memory state.

Main Path

- ▶ Start from the question: What is the role of Value Representation and Data Objects in the course story?
- ▶ Build the model: Value Representation and Data Objects is one step in the path from source text to reliable execution.
- ▶ End with a checkable action: Students can connect the topic to a concrete command, program state, or memory state.
- ▶ Keep only this slide on the horizontal path during the live lecture.

Value representation

- Hexadecimal numbers begin with 0x
- 0xc23 is 2316 (which is 36 in the base of 10)
- Octal numbers begin with 0.

#150

Worked Example

- ▶ Use this as the live trace: Use one short trace, then ask students which state changed and which state did not.
- ▶ Representative source slide: 151
- ▶ Ask students to predict the next state before revealing the explanation.
- ▶ Then connect the result back to the core claim.

Data object definition

- Syntax:
- type identifier;
- Function:
- Reserve in the memory a part equal to the size required to store a data object of this type
- Associate the starting address of this memory part with the identifier

#151

Anecdote Or Motivation

Keep the discussion anchored in observable behavior: command output, compiler message, or memory drawing.

- ▶ Use this only if students need a reason to care.
- ▶ If the room is already following, skip down to the check question.

Check Question

- ▶ Ask for a prediction, not a definition.
- ▶ Require a short justification: command trace, memory drawing, type rule, or file state.
- ▶ If more than a third of the room hesitates, go down to the source backup.

Defining primitive data objects: Examples

```
Definitions
int i;
float f1, f2;
unsigned char c1, c2;
Definitions with initialization
int i = 3;
float f1 = 1.2, f2 = 0.3;
unsigned char c1 = 'X', c2 = 65;
```

-- What is value of these identifiers?
-- Whatever is already stored in the memory just reserved.
-- (except global and static variables)

- INITIALIZE ALL VARIABLES !!

- 00

- Value of ASCII character

#156

Backup Index

Original slides 150-156

Value representation

- Hexadecimal numbers begin with 0x:
- 0x23 is 23₁₆ (which is 35 in the base of 10)
- Octal numbers begin with 0:

Backup: Value representation

Original slide 150

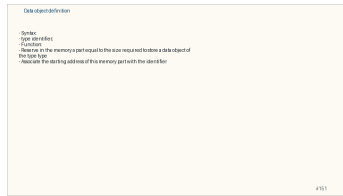
- ▶ Hexadecimal numbers begin with 0x:
- ▶ 0x23 is 23₁₆ (which is 35 in the base of 10)
- ▶ Octal numbers begin with 0:
- ▶ 023 is 23₈ (which is 19 in the base of 10)
- ▶ No prefix for decimal numbers:
- ▶ 23 is 23₁₀
- ▶ Negative representation - operator
- ▶ -23 is the negation of 23



Backup: Data object definition

Original slide 151

- ▶ Syntax:
- ▶ type identifier;
- ▶ Function:
- ▶ Reserve in the memory a part equal to the size required to store a data object of the type type
- ▶ Associate the starting address of this memory part with the identifier



Backup: Primitive data types: Type specifiers

Original slide 152

- ▶ Same as in JAVA:
- ▶ Integral
- ▶ char, short, int, long
- ▶ Real and complex
- ▶ float, double
- ▶ With additional specifiers for the representation:
- ▶ unsigned: always 0 or positive
- ▶ Representation: Binary
- ▶ signed: positive or negative:
- ▶ Representation: implementation depended
- ▶ 1's complement
- ▶ 2's complement (usual one)
- ▶ sign and magnitude
- ▶ Example: unsigned short int

Primitive data types: Type specifiers

- Same as in JAVA
- Integral
- char, short, int, long
- Real and complex
- float, double
- With additional specifiers for the representation
- unsigned: always 0 or positive
- Representation: Binary
- signed: positive or negative
- Representation: implementation depended
- 1's complement
- 2's complement (usual one)
- sign and magnitude
- Example: unsigned short int

#152

Backup: Primitive data types: Size

Original slide 153

The exact data type size is NOT part of the C standard
int can be 16 or 32 bits...

The standard only says that the range of a larger type is at least as big as the smaller type

char short long long long

char: The minimum supported data type

Compiler decides based on the target platform architecture! (though, there is a minimum based on the standard)

sizeof() can give the actual size in bytes

To give a specific storage size use the YintX_t type

Y = u for unsigned

X = 8, 16, 32 etc for the number of bits to be used



Backup: Primitive data types: Min values

Original slide 154

Type	Storage size	Value range
char	usually 1 byte	signed (but not standard)
unsigned char	1 byte	0 to 255
signed char	1 byte	(-128) to 127
int	2 bytes	(-32768) to 32767
(usually 4 bytes)		
unsigned int	2 bytes	0 to 65535
(usually 4 bytes)		
short	2 bytes	(-32768) to 32767
unsigned short	2 bytes	0 to 65535
long	4 bytes	(-2147483648) to 2147483647
unsigned long	4 bytes	0 to 4294967295
long long	8 bytes	(-9223372036854775808) to 9223372036854775807
unsigned long long	8 bytes	0 to 18446744073709551615

Primitive data types: Min values
Type Storage size Value range char usually signed (but not standard) unsigned char 1 byte 0 to 255 signed char 1 byte (-128) to 127 int 2 bytes (-32768) to 32767 (usually 4 bytes) unsigned int 2 bytes 0 to 65535 (usually 4 bytes) short 2 bytes (-32768) to 32767 unsigned short 2 bytes 0 to 65535 long 4 bytes (-2147483648) to 2147483647 unsigned long 4 bytes 0 to 4294967295 long long 8 bytes (-9223372036854775808) to 9223372036854775807 unsigned long long 8 bytes 0 to 18446744073709551615

Backup: Primitive data types: Float

Original slide 155

- ▶ Type Storage size Value range Precision
- ▶ float 4 byte $1.2\text{E}-38$ to $3.4\text{E}+38$ single
- ▶ double 8 byte $2.3\text{E}-308$ to $1.7\text{E}+308$ double
- ▶ long double 10 byte $3.4\text{E}-4932$ to $1.1\text{E}+4932$ extended

Primitive data types: Float				
Type	Storage size	Value range	Precision	
Real	4 byte	$1.2\text{E}-38$ to $3.4\text{E}+38$	single	
double	8 byte	$2.3\text{E}-308$ to $1.7\text{E}+308$	double	
long double	10 byte	$3.4\text{E}-4932$ to $1.1\text{E}+4932$	extended	

Backup: Defining primitive data objects: Examples

Original slide 156

Definitions

```
int i;
```

```
float f1, f2;
```

```
unsigned char c1, c2;
```

Definitions with initialization

```
int i = 3;
```

```
float f1 = 1.2, f2 = 0.3;
```

```
unsigned char c1 = 'A', c2 = 65;
```

- ▶ ▶ What is value of these identifiers?
- ▶ ▶ Whatever is already stored in the memory just reserved...
- ▶ (except global and static variables)
- ▶ INITIALIZE ALL VARIABLES !!
- ▶ !!!
- ▶ Value of ASCII character



Operators

Operators

Live pacing: about 8 min

Question. What exactly is stored when a C variable has a type?

Core claim. A C type is a contract about size, interpretation, operations, conversion, and lifetime.

Target capability. Students can predict simple operator results and recognize dangerous implicit conversions.

Main Path

- ▶ Start from the question: What exactly is stored when a C variable has a type?
- ▶ Build the model: A C type is a contract about size, interpretation, operations, conversion, and lifetime.
- ▶ End with a checkable action: Students can predict simple operator results and recognize dangerous implicit conversions.
- ▶ Keep only this slide on the horizontal path during the live lecture.

Operators
Perform operations on data objects: They have priorities in the order they are applied: Check the priority order USE PARENTHESIS
Operator types
Arithmetic
Relational
Logical
Bitwise
Assignment
...

#157

Worked Example

- ▶ Use this as the live trace: Use `int`, `unsigned`, and `float` values that print differently under `%d`, `%u`, and `%f`.
- ▶ Representative source slide: 158
- ▶ Ask students to predict the next state before revealing the explanation.
- ▶ Then connect the result back to the core claim.

Arithmetic operators	
	- A = 10, B = 20
Operator Description Example + Adds two operands. A + B = 30 - Subtracts second operand from the first. A - B = -10 * Multiplies both operands. A * B = 200 / Divides numerator by denominator. B / A = 2 % Remainder after integer division. B % A = 0 (assignment at the same time) ++ Increases the integer value by one. A++ = 11 -- Decreases the integer value by one. A-- = 9	
-	
Prefix (Post-increment): Do the operation and return old value A++ returns 10 and then does A=11 Prefix (Pre-increment): Do the operation and return new value ++A does A=11 and then returns A	
#158	

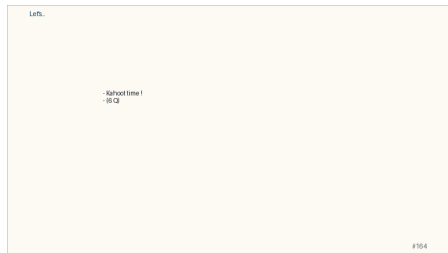
Anecdote Or Motivation

The machine stores bits; the type tells the compiler and programmer how to read those bits.

- ▶ Use this only if students need a reason to care.
- ▶ If the room is already following, skip down to the check question.

Check Question

- ▶ Ask for a prediction, not a definition.
- ▶ Require a short justification: command trace, memory drawing, type rule, or file state.
- ▶ If more than a third of the room hesitates, go down to the source backup.



Backup Index

Original slides 157-164

Operators

Perform operations on data objects

They have priorities in the order they are applied:

Check the priority order

USE PARENTHESIS

Operator types

Arithmetic

Relational

Logical

Bitwise

Assignment

.....

Backup: Operators

Original slide 157

Perform operations on data objects
They have priorities in the order they are applied
Check the priority order
USE PARENTHESIS
Operator types
Arithmetic
Relational
Logical
Bitwise
Assignment
.....

Operators
Perform operations on data objects They have priorities in the order they are applied Check the priority order USE PARENTHESIS Operator types Arithmetic Relational Logical Bitwise Assignment
#157

Backup: Arithmetic operators

Original slide 158

► $A = 10, B = 20$

Operator Description Example

+ Adds two operands. $A + B = 30$

- Subtracts second operand from the first. $A - B = -10$

* Multiplies both operands. $A * B = 200$

/ Divides numerator by de-numerator. $B / A = 2$

% Remainder after integer division. $B \% A = 0$

(assignment at the same time).

++ Increases the integer value by one $A++ = 11$

-- Decreases the integer value by one. $A-- = 9$

► !!!

Postfix (Post-increment): Do the operation and return old value

$A++$ returns 10 and then does $A=11$

Prefix (Pre-increment): Do the operation and return new value

$++A$ does $=11$ and then returns A

Arithmetic operators	
$-A = 10, B = 20$	
Operator Description Example	
+ Adds two operands. $A + B = 30$	
- Subtracts second operand from the first. $A - B = -10$	
* Multiplies both operands. $A * B = 200$	
/ Divides numerator by de-numerator. $B / A = 2$	
% Remainder after integer division. $B \% A = 0$	
(assignment at the same time).	
++ Increases the integer value by one. $A++ = 11$	
-- Decreases the integer value by one. $A-- = 9$	
-11	
Postfix (Post-increment): Do the operation and return old value. $A++$ returns 10 and then does $A=11$ Prefix (Pre-increment): Do the operation and return new value $++A$ does $=11$ and then returns A	
#158	

Backup: Relational operators

Original slide 159

► A = 10, B = 20

Op. Description

`==` Checks if the values of two operands are equal.

If yes, the condition becomes **true**.

`!=` Checks if the values of two operands are not equal.
different. If yes, the condition becomes **true**.

`>` Checks if the value of left operand is greater than the value of right operand. If yes, it becomes **true**.

`<` Checks if the value of left operand is less than the value of right operand. If yes, it becomes **true**.

`>=` Checks if the value of left operand is greater than or equal to the value of right operand.

`<=` Checks if the value of left operand is less than or equal to the value of right operand.

► Example

Relational operators	
Op. Description	Example
<code>==</code> Checks if the values of two operands are equal. If yes, the condition becomes true.	<code>A = 10, B = 20</code> <code>(A == B)</code> is not true
<code>!=</code> Checks if the values of two operands are not equal. If yes, the condition becomes true.	<code>(A != B)</code> is true
<code>></code> Checks if the value of left operand is greater than the value of right operand. If yes, it becomes true.	<code>(A > B)</code> is not true
<code><</code> Checks if the value of left operand is less than the value of right operand. If yes, it becomes true.	<code>(A < B)</code> is true
<code>>=</code> Checks if the value of left operand is greater than or equal to the value of right operand.	
<code><=</code> Checks if the value of left operand is less than or equal to the value of right operand.	
Any integer value can be considered as Boolean	
A FALSE expression is coded as a zero integer	
A TRUE expression is coded as any non-zero integer value	

#159

Backup: Logical operators

Original slide 160

- ▶ The operands are TRUE or FALSE
- ▶ $A = 1, B = 0$
- ▶ Example
- ▶ $(A \ \&\& \ B)$ is false.
- ▶ $(A \ || \ B)$ is true.
- ▶ $!(A \ \&\& \ B)$ is true.

Op.

$\&\&$

$||$

!

- ▶ Description
- ▶ Logical AND: If both the operands are non-zero, then the condition becomes true.
- ▶ Does not evaluate 2nd operand, if the first is false
- ▶ Logical OR: If any of the two operands is non-zero,

Logical operators

The operands TRUE or FALSE

- $A = 1, B = 0$

Op.

Description

Logical AND: If both the operands are non-zero, then the condition becomes true.

Does not evaluate 2nd operand, if the first is false.

Logical OR: If any of the two operands is non-zero, then the condition becomes true.

Does not evaluate 2nd operand, if the first is true.

Logical NOT: It is used to reverse the logical state of its operand. If a condition is true, then Logical NOT operator will make it false.

Example

$(A \ \&\& \ B)$ is false.

$(A \ || \ B)$ is true.

$!(A \ \&\& \ B)$ is true.

#160

Backup: Bitwise operators

Original slide 161

- ▶ The operands are integral (short, int, unsigned, etc)
- ▶ $A = 0x3$ (0011) $B = 0x2$ (0010)
- ▶ Op. Description Example
- ▶ Binary AND: Copies a bit to the result, if it exists in both operands.
- ▶ Binary OR: Copies a bit if it exists in either operand.
- ▶ Binary XOR: Copies the bit if it is set only in one operand but not both.
- ▶ Binary Ones Complement: Has the effect of 'flipping' bits.
- ▶ Binary Left Shift: The left operands value is moved left by the number of bits specified by the right operand.
- ▶ Binary Right Shift: The left operands value is moved right by the number of bits specified by the right operand.
- ▶ $(A \& B) = 0x2$

Bitwise operators		
The operands are integral (short, int, unsigned, etc)		
Op. Description Example		
&	Binary AND: Copies a bit to the result, if it exists in both operands.	$A = 0x3$ (0011) $B = 0x2$ (0010) - 0011 - 0010 ----- - 0010
	Binary OR: Copies a bit if it exists in either operand.	- 1A B = 0x3 - 0011
^	Binary XOR: Copies the bit if it is set only in one operand but not both.	- 1A ^ B = 0x1 - 0001
~	Binary Ones Complement: Has the effect of flipping bits.	- ~A = 0x4 - 1100
<<	Binary Left Shift: The left operand's value is moved left by the number of bits specified by the right operand.	- (A << 2) = 0x12 - 1100
>>	Binary Right Shift: The left operand's value is moved right by the number of bits specified by the right operand.	- (A >> 2) = 0x03 - 0000

Backup: Assignment operators

Original slide 162

- ▶ $A = 10, B = 20$
- ▶ Op. Description Example
- ▶ Simple assignment operator. Assigns values from right side operands to left side operand
- ▶ In can be combined with the other arithmetic operators (+, -, *, /, %) and bitwise operators (&, |, ^, «, »).
- ▶ It performs the arithmetic operation with right operand and the left operand and assign the result to the left operand.
- ▶ $C = A + B$
- ▶ $C = 30$

Assignment operators

Op. Description Example

assignment operator. Assigns values from right side to left side operand combined with the other arithmetic operators (+, -, *, /, %, &, |, ^, «, ») to the arithmetic operation with right operand and the left operand and assign the result to the left operand.

$A = 10, B = 20$

$C = A + B$
 $C = 30$

$A = 1$
 $A = 11$

#162

=
+=
-=
*=
/=

Backup: Common error: Assignment Vs Testing

Original slide 163

- ▶ Assignment: `nom_variable = expression ;`
- ▶ `x=3;`
- ▶ It is an expression that stores the value 3 to the variable `x`
- ▶ Test of equality: `expression1 == expression2 ;`
- ▶ `x==3;`
- ▶ It returns 1 if the stored value in `x` is equal to 3, otherwise 0
- ▶ ATTENTION:
- ▶ `=` and `==` are different !

Common error: Assignment vs. Testing

- Assignment: `nom_variable = expression ;`
 - `x=3;`
 - It is an expression that stores the value 3 to the variable `x`

ATTENTION

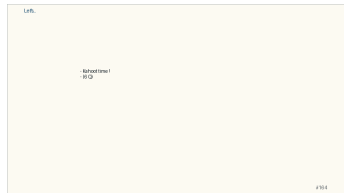
- `=` and `==` are different !

#163

Backup: Let's...

Original slide 164

- ▶ Kahoot time !
- ▶ (6 Q)



Conversions

Conversions

Live pacing: about 8 min

Question. What exactly is stored when a C variable has a type?

Core claim. A C type is a contract about size, interpretation, operations, conversion, and lifetime.

Target capability. Students can predict simple operator results and recognize dangerous implicit conversions.

Main Path

- ▶ Start from the question: What exactly is stored when a C variable has a type?
- ▶ Build the model: A C type is a contract about size, interpretation, operations, conversion, and lifetime.
- ▶ End with a checkable action: Students can predict simple operator results and recognize dangerous implicit conversions.
- ▶ Keep only this slide on the horizontal path during the live lecture.

So what happens if we mix types in an operation?

- a (operator) b type a = type b

- Automatic IMPLICIT and SILENT conversion!
- Both operands (a, b) must be data objects of the same type
- Converted to the common higher rank of the operands based on the type width:
- PROMOTION
- Integer promotions:
signed → unsigned
small integer types char, short, int8, int8 etc (BOTH signed and unsigned) are converted to
- int: IF the range of int can represent ALL values of the original type
else: unsigned int

#165

Worked Example

- ▶ Use this as the live trace: Use `int`, `unsigned`, and `float` values that print differently under `%d`, `%u`, and `%f`.
- ▶ Representative source slide: 174
- ▶ Ask students to predict the next state before revealing the explanation.
- ▶ Then connect the result back to the core claim.

Type conversion: Usual problems

Casting a larger type to a smaller one (DEMONSTRATION):
Truncation: Cuts off the MSB (overflow occurs).
`int i = 0x12345678;
short int g;
g = i;
Comparisons, especially with unsigned, char and short!
unsigned int a = 0;
if (a > 1)
 printf("TRUE\n");
else
 printf("FALSE\n");`

- SECURITY ISSUES

#174

Anecdote Or Motivation

The machine stores bits; the type tells the compiler and programmer how to read those bits.

- ▶ Use this only if students need a reason to care.
- ▶ If the room is already following, skip down to the check question.

Check Question

- ▶ Ask for a prediction, not a definition.
- ▶ Require a short justification: command trace, memory drawing, type rule, or file state.
- ▶ If more than a third of the room hesitates, go down to the source backup.

Type conversion: Usual problems

Casting a larger type to a smaller one (DEMONSTRATION):
Truncation: Cutoff the MSB (overflow occurs):
int i = 0x12345678;
short int j;
j = i;
Comparisons, especially with unsigned, char and short!
unsigned int a = 0;
if (a > -1)
printf("TRUE\n");
else
printf("FALSE\n");

- SECURITY ISSUES

#174

Backup Index

Original slides 165-174

So, what happens if we mix types in an operation?

- a (operator) b type a = type b

- Automatic IMPLICIT and SILENT conversion!
- Both operands (a, b) must be data objects of the same type
- Converted to the common higher rank of the operands based on the type width:
- PROMOTION
- Integer promotions:
 - signed -> unsigned
 - small integer types: char, short, int16, int8 etc (BOTH signed and unsigned) are converted to int: IF the range of int can represent ALL values of the original type
 - else: unsigned int

Backup: So, what happens if we mix types in an operation?

Original slide 165

- ▶ a (operator) b type a = type b
- ▶ Automatic IMPLICIT and SILENT conversion!
- ▶ Both operands (a, b) must be data objects of the same type
- ▶ Converted to the common higher rank of the operands based on the type width:
- ▶ PROMOTION
- ▶ Integer promotions:
- ▶ signed $\rightarrow$ unsigned
- ▶ small integer types: char, short, int16, int8 etc (BOTH signed and unsigned) are converted to
- ▶ int : IF the range of int can represent ALL values of the original type
- ▶ else: unsigned int

So, what happens if we mix types in an operation?

-> a (operator) b type a = type b

- Automatic IMPLICIT and SILENT conversion!

- Both operands (a, b) must be data objects of the same type

- Converted to the common higher rank of the operands based on the type width:

PROMOTION

Integer promotions

signed $\rightarrow$ unsigned

small integer types: char, short, int16, int8 etc (BOTH signed and unsigned) are converted to

int: IF the range of int can represent ALL values of the original type

else: unsigned int

#165

Backup: And during assignment?

Original slide 166

- ▶ $c = a \text{ (operator) } b$
- ▶ $\text{type } c = \text{type (result of operation)}$
- ▶Still automatic IMPLICIT and SILENT conversion!
- ▶ Converted to the type of data object that stores the result:
- ▶ Higher rank: PROMOTION
- ▶ Lower rank: DEMOTION

And during assignment?

$c = a \text{ (operator) } b$
 $\text{type } c = \text{type (result of operation)}$

... Still automatic IMPLICIT and SILENT conversion!
(Converted to the type of data object that stores the result)

Higher rank: PROMOTION
Lower rank: DEMOTION

#100

Backup: Primitive data types: Implicit conversion

Original slide 167

- ▶ long double
- ▶ double
- ▶ float
- ▶ unsigned long long
- ▶ long long
- ▶ DEMOTION
- ▶ PROMOTION
- ▶ unsigned long
- ▶ long
- ▶ unsigned int
- ▶ int
- ▶ Small integer types
- ▶ short
- ▶ char
- ▶ 2 Bytes
- ▶ 1 Byte



Backup: Explicit conversion: Cast operator

Original slide 168

- ▶ Syntax
- ▶ (type) identifier;
- ▶ Creates a copy of the data object and changes its type.
- ▶ The data object of the identifier is NOT modified

Explicit conversion: Cast operator

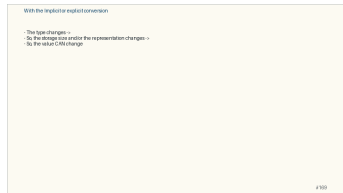
- Syntax
- (type) identifier;
- Creates a copy of the data object and changes its type
- The data object of the identifier is NOT modified

#168

Backup: With the Implicit or explicit conversion

Original slide 169

- ▶ The type changes ->
- ▶ So, the storage size and/or the representation changes
->
- ▶ So, the value CAN change



Backup: And, when we cast different representation ?

Original slide 170

- ▶ Casting float to int: It changes representation!
- ▶ float is represented in IEEE 754 single precision
- ▶ int is usually 2's Complement
- ▶ float $f = 3.9$;
- ▶ `int val = (int) f ;`
- ▶ Value gets truncated !
- ▶ 3.9 is truncated to 3.0 and stored as 3 in 2's complement
- ▶ Casting int to float: It changes representation !
- ▶ `int f = 3 ;`
- ▶ `float val = (float) f ;`
- ▶ Value to convert to float (thus, 3 is converted to 3.0)

And, when we cast different representation ?

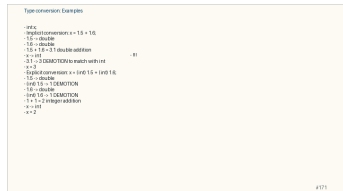
- Casting float to int: It changes representation !
- float is represented in IEEE 754 single precision
- int is usually 2's Complement
- float $f = 3.9$;
- `int val = (int) f ;`
- Value gets truncated !
- 3.9 is truncated to 3.0 and stored as 3 in 2's complement
- Casting int to float: It changes representation !
- `int f = 3 ;`
- `float val = (float) f ;`
- Value to convert to float (thus, 3 is converted to 3.0)

#170

Backup: Type conversion: Examples

Original slide 171

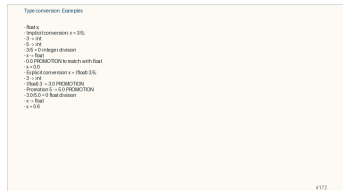
- ▶ `int x;`
- ▶ Implicit conversion: `x = 1.5 + 1.6;`
- ▶ `1.5 -> double`
- ▶ `1.6 -> double`
- ▶ `1.5 + 1.6 = 3.1` double addition
- ▶ `x -> int`
- ▶ `3.1 -> 3` DEMOTION to match with `int`
- ▶ `x = 3`
- ▶ Explicit conversion: `x = (int) 1.5 + (int) 1.6;`
- ▶ `1.5 -> double`
- ▶ `(int) 1.5 -> 1` DEMOTION
- ▶ `1.6 -> double`
- ▶ `(int) 1.6 -> 1` DEMOTION
- ▶ `1 + 1 = 2` integer addition
- ▶ `x -> int`
- ▶ `x = 2`



Backup: Type conversion: Examples

Original slide 172

- ▶ `float x;`
- ▶ Implicit conversion: `x = 3/5;`
- ▶ `3 -> int`
- ▶ `5 -> int`
- ▶ `3/5 = 0` integer division
- ▶ `x -> float`
- ▶ `0.0 PROMOTION` to match with float
- ▶ `x = 0.0`
- ▶ Explicit conversion `x = (float) 3/5;`
- ▶ `3 -> int`
- ▶ `(float) 3 -> 3.0 PROMOTION`
- ▶ `Promotion 5 -> 5.0 PROMOTION`
- ▶ `3.0/5.0 = 0.6` float division
- ▶ `x -> float`
- ▶ `x = 0.6`



Backup: Type conversion: Usual problems

Original slide 173

Casting signed to unsigned:

Negative value is converted to a high positive value

Especially occurs during function calls or return values

Casting a smaller signed int type to a larger unsigned

Sign extension: The sign bit is used in the unused bits

char and short are signed!

Negative value is converted to a high positive value

► SECURITY ISSUES

Type conversion: Usual problems

Casting signed to unsigned:
Negative value is converted to a high positive value.
Especially occurs during function calls or return values.
Casting smaller signed int type to a larger unsigned int type.
Sign extension: The sign bit is used in the unused bits, i.e. the Most Significant Bit (MSB) of the target type.
char and short are signed!
Negative value is converted to a high positive value.

SECURITY ISSUES

#173

Backup: Type conversion: Usual problems

Original slide 174

Casting a larger type to a smaller one (DEMOTION):
Truncation: Cuts off the MSB (overflow occurs).

```
int f = 0x12345678 ;
```

```
short int g;
```

```
g = f;
```

Comparisons, especially with **unsigned**, **char** and **short** !

```
unsigned int a = 0 ;
```

```
if (a > -1)
```

```
printf("TRUE\n");
```

```
else
```

```
printf("FALSE\n");
```

► SECURITY ISSUES

Type conversion: Usual problems

Casting a larger type to a smaller one (DEMOTION):
Truncation: Cuts off the MSB (overflow occurs).
int f = 0x12345678;
short int g;
g = f;
Comparisons, especially with unsigned, char and short!
unsigned int a = 0;
if (a > -1)
printf("TRUE\n");
else
printf("FALSE\n");

- SECURITY ISSUES

#174

Allocation, Qualifiers, Macros

Allocation, Qualifiers, Macros

Live pacing: about 8 min

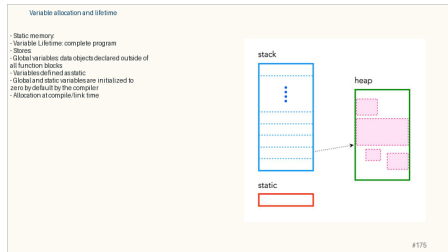
Question. What does the operating system do between the program and the machine?

Core claim. The OS gives a program controlled access to files, processes, memory, and devices.

Target capability. Students can place shell commands, processes, files, and permissions in the OS layer.

Main Path

- ▶ Start from the question: What does the operating system do between the program and the machine?
- ▶ Build the model: The OS gives a program controlled access to files, processes, memory, and devices.
- ▶ End with a checkable action: Students can place shell commands, processes, files, and permissions in the OS layer.
- ▶ Keep only this slide on the horizontal path during the live lecture.



Worked Example

- ▶ Use this as the live trace: Compare `ls`, an editor, and a compiled program: all are processes asking the OS for services.
- ▶ Representative source slide: 179
- ▶ Ask students to predict the next state before revealing the explanation.
- ▶ Then connect the result back to the core claim.

Object like Macros

- Syntax: Through pre-processor directive `#define`
- Adeline: IDENTIFIER value
- Function:
 - Gives a symbolic name to a data object
- Convention: Symbolic names are given in UPPERCASE
- Examples
 - Adeline P1.3.14.16
 - Adeline NCM.100

#179

Anecdote Or Motivation

A useful mental image: user programs are guests; the OS is the host that owns the keys.

- ▶ Use this only if students need a reason to care.
- ▶ If the room is already following, skip down to the check question.

Check Question

- ▶ Ask for a prediction, not a definition.
- ▶ Require a short justification: command trace, memory drawing, type rule, or file state.
- ▶ If more than a third of the room hesitates, go down to the source backup.

Other reserved constants

- Escape sequences
 - The character '\n' is a new line (on Unix systems)
 - The character '\b' is a backspace
 - The character '\t' is a tab space
 - The character '\0' is null character
- The double quotes ... defines a string of characters
- There is NO string type in C
- String = array of characters that ends with the null character '\0'
- The single quote '...' defines a character

#180

Backup Index

Original slides 175-180

Variable allocation and lifetime

- Static memory:
- Variable Lifetime: complete program
- Stores:
 - Global variables: data objects declared outside of all function blocks
 - Variables defined as static
 - Global and static variables are initialized to zero by default by the compiler
 - Allocation at compile/link time

stack



heap



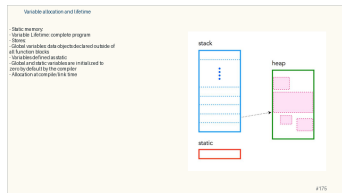
static



Backup: Variable allocation and lifetime

Original slide 175

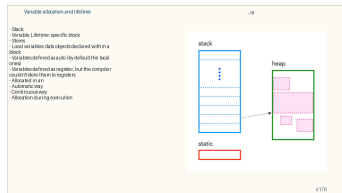
- ▶ Static memory:
- ▶ Variable Lifetime: complete program
- ▶ Stores:
- ▶ Global variables: data objects declared outside of all function blocks
- ▶ Variables defined as static
- ▶ Global and static variables are initialized to zero by default by the compiler
- ▶ Allocation at compile/link time



Backup: Variable allocation and lifetime

Original slide 176

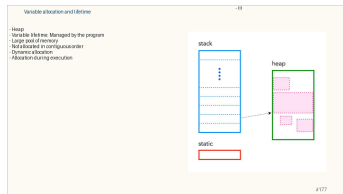
- ▶ !!!
- ▶ Stack:
- ▶ Variable Lifetime: specific block
- ▶ Stores
- ▶ Local variables: data objects declared within a block
- ▶ Variables defined as auto (by default the local ones)
- ▶ Variables defined as register, but the compiler couldn't store them to registers
- ▶ Allocated in an
- ▶ Automatic way
- ▶ Continuous way
- ▶ Allocation during execution



Backup: Variable allocation and lifetime

Original slide 177

- ▶ !!!
- ▶ Heap:
- ▶ Variable lifetime: Managed by the program
- ▶ Large pool of memory
- ▶ Not allocated in contiguous order
- ▶ Dynamic allocation
- ▶ Allocation during execution



Backup: Data objects: Qualifiers

Original slide 178

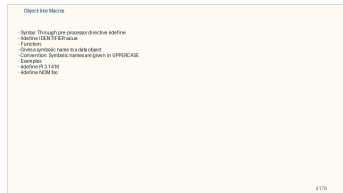
- ▶ Express additional information about a value
- ▶ Ensure the correct use of the data object
- ▶ `const`: The value of the data object does not change once it has been initialized
- ▶ `volatile`: The compiler does not optimize, the data object is always accessed from the memory
- ▶ `restrict`: The only way to access a data object is through the pointer that points to it



Backup: Object-like Macros

Original slide 179

- ▶ Syntax: Through pre-processor directive `#define`
- ▶ `#define IDENTIFIER value`
- ▶ Function:
- ▶ Gives a symbolic name to a data object
- ▶ Convention: Symbolic names are given in UPPERCASE
- ▶ Examples
- ▶ `#define PI 3.1416`
- ▶ `#define NOM foo`



Backup: Other reserved constants

Original slide 180

- ▶ Escape sequences:
- ▶ The character `'\n'` is a new line (on Unix systems)
- ▶ The character `'\b'` is a backspace
- ▶ The character `'\t'` is a tab space
- ▶ The character `'\0'` is null character
- ▶ The double quotes `...` defines a string of characters
- ▶ There is NO string type in C
- ▶ String = array of characters that ends with the null character (`\0`)
- ▶ The simple quote `'.'` defines a character

